Machine Trans

```
execute_util 에서 텍스트, 링크, 이미지를 가져옵니다 .
사실 에서 a100_flop_per_sec, h100_flop_per_sec를 가져옵니다 .
import torch.n ional을 F 로 가
timeit 가져오기
수입 토치
import Iterable 입력 하
토치 수입 nn 에서
numpy를 np 로 가져오기
lecture_util 에서 article_link를 가져옵니다 .
jaxtyping 에서 Float 가져오기
einops 에서 가져오기 재배열, einsum, reduce
참조 에서 zero_2019 가져오기


def main():
```

마지막 강의: 개요, 토큰화

이 강의 개요:

- 모델을 학습하는 데 필요한 모든 기본 요소에 대해 논의해 보겠습니다.
- 우리는 텐서에서 모델, 옵티마이저, 그리고 학습 루프까지 하향식으로 살펴볼 것입니다.
- 우리는 효율성(자원 활용)에 많은 주의를 기울일 것입니다.

특히, 우리는 두 가지 유형의 리소스를 설명할 것입니다.

- 메모리(GB)
- 계산(FLOP)

동기부여 질문()

우리는 트랜스포머에 대해서는 다루지 않을 것입니다.
훌륭한 설명이 있습니다.
과제 1 핸드아웃
수학적 설명
일러스트 트랜스포머
그림으로 표현된 GPT-2
대신, 우리는 더 간단한 모델을 사용해 작업할 것입니다.

어떤 지식을 얻을 수 있나요?

- 역학: 간단함(PyTorch만 있으면 됨)
- 사고방식: 자원 회계(반드시 기억하세요)
- 직관: 대략적인 내용(대규모 모델 없음)

메모리 회계

텐서_기본()
텐서_메모리()

회계 계산

GPU의 텐서()
텐서 연산()
텐서_이노프스()
텐서_연산_플롭()
그라디언트_기본()
그라디언트_플롭()

모델

모듈_매개변수()
커스텀_모델()

훈련 루프와 모범 사례
무작위성에 대한 메모()
데이터 로딩()

옵티마이저()
train_loop()
체크포인팅()
혼합_정밀_훈련()

```
def 동기부여 질문():
```

냅킨으로 계산해 봅시다.

질문: 1024개의 H100에서 15T 토큰으로 70B 매개변수 모델을 훈련하는 데 얼마나 걸릴까요?

```
총 플롭스 = 6 * 70e9 * 15e12 # @총 플롭스 검사
h100_flop_per_sec == 1979e12 / 2 를 주장합니다 .
손실 = 0.5
flops_per_day = h100_flop_per_sec * mfu * 1024 * 60 * 60 * 24 # @inspect flops_per_day
일수 = 총_실패수 / 일당_실패수 # @inspect 일수
```

질문: AdamW를 사용하여 8개의 H100으로 훈련할 수 있는 가장 큰 모델은 무엇입니까(단순히)?

```
h100_bytes = 80e9 # @h100_bytes를 검사합니다.
bytes_per_parameter = 4 + 4 + (4 + 4) # 매개변수, 그래디언트, 최적화 상태 @inspect bytes_per_parameter
num_parameters = (h100_bytes * 8) / bytes_per_parameter # @inspect num_parameters
```

주의 사항 1: 매개변수와 그래디언트에 float32를 사용하고 있습니다. 매개변수에 bf16을 사용할 수도 있습니다.
그리고 그래디언트(2 + 2)를 사용하고 매개변수(4)의 추가 float32 복사본을 유지합니다. 이렇게 하면 메모리가 절약되지는 않지만 더 빠르게. [라즈반다라+ 2019]
주의 사항 2: 활성화는 고려되지 않습니다(배치 크기 및 시퀀스 길이에 따라 다름).

이는 대략적인 계산입니다.

```
def tensors_basics():
```

텐서는 매개변수, 그래디언트, 최적화 상태, 데이터 등 모든 것을 저장하는 기본 구성 요소입니다.
활성화.
[텐서에 대한 PyTorch 문서]

텐서는 여러 가지 방법으로 생성할 수 있습니다.

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]]) # @x 검사
x = torch.zeros(4, 8) # 모두 0인 4x8 행렬 @inspect x
x = torch.ones(4, 8) # 모두 1인 4x8 행렬 @inspect x
x = torch.randn(4, 8) # iid Normal(0, 1) 샘플의 4x8 행렬 @inspect x
```

값을 할당하지만 초기화하지 마세요.

```
x = torch.empty(4, 8) # 초기화되지 않은 값의 4x8 행렬 @inspect x
```

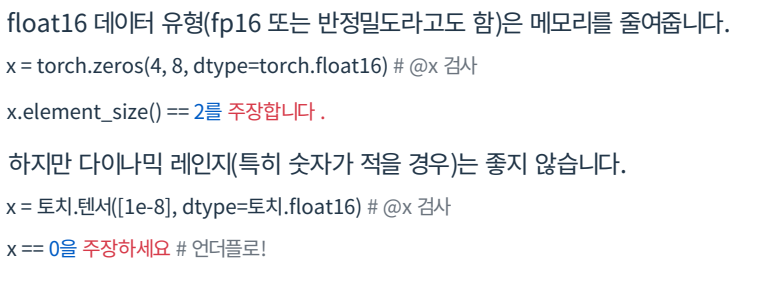
...나중에 값을 설정하기 위해 사용자 정의 논리를 사용하고 싶기 때문입니다.

```
nn.init.trunc_normal_(x, 평균=0, 표준편차=1, a=-2, b=2) # @x 검사


def 텐서_메모리():
```

거의 모든 것(매개변수, 그래디언트, 활성화, 최적화 상태)은 부동 소수점 숫자로 저장됩니다.

플로트32

[위키피디아]



float32 데이터 유형(fp32 또는 단정도라고도 함)이 기본값입니다.
전통적으로 과학 컴퓨팅에서는 float32가 기준입니다. 일부에서는 double precision(float64)을 사용할 수 있습니다.
사례.
딥러닝에서는 훨씬 더 엉성해질 수 있습니다.

이 텐서의 메모리 사용량을 살펴보겠습니다.
메모리는 (i) 값의 개수와 (ii) 각 값의 데이터 유형에 따라 결정됩니다.

```
x = 토치.zeros(4, 8) # @x 검사
assert x.dtype == torch.float32 # 기본 유형
x.numel() == 4 * 8을 주장합니다 .
assert x.element_size() == 4 # Float은 4바이트입니다
get_memory_usage(x) == 4 * 8 * 4 # 128바이트를 주장합니다 .
```

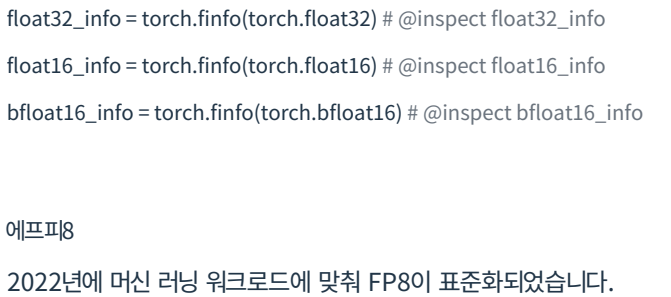
GPT-3의 피드포워드 계층에 있는 하나의 매트릭스:

```
assert get_memory_usage(torch.empty(12288 * 4, 12288)) == 2304 * 1024 * 1024 # 2.3GB
```

...정말 많은 양이죠!

플로트16

[위키피디아]



float16 데이터 유형(fp16 또는 반정밀도라고도 함)은 메모리를 줄여줍니다.

```
x = torch.zeros(4, 8, dtype=torch.float16) # @x 검사
x.element_size() == 2를 주장합니다 .
```

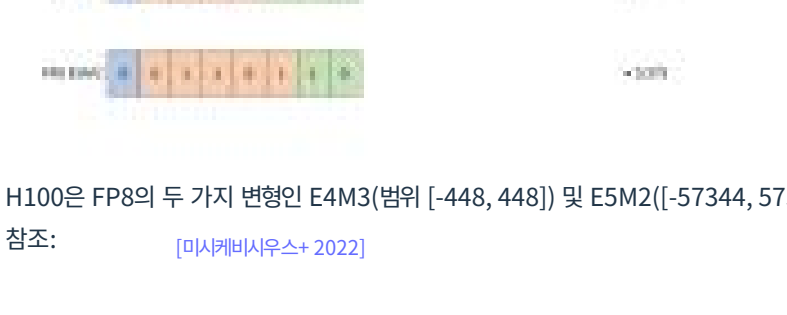
하지만 다이나믹 레인지(특히 숫자가 적을 경우)는 좋지 않습니다.

```
x = 토치.텐서([1e-8], dtype=토치.float16) # @x 검사
x == 0을 주장하세요 # 언더플로!
```

훈련 중에 이런 일이 발생하면 불안정해질 수 있습니다.

비플로트16

[위키피디아]



Google Brain은 이 문제를 해결하기 위해 2018년에 bfloat(Brain Floating Point)를 개발했습니다.
bfloat16은 float16과 동일한 메모리를 사용하지만 float32와 동일한 동적 범위를 갖습니다!
유일한 문제점은 해상도가 나쁘다는 점인데, 이는 딥러닝에서는 그다지 문제가 되지 않습니다.

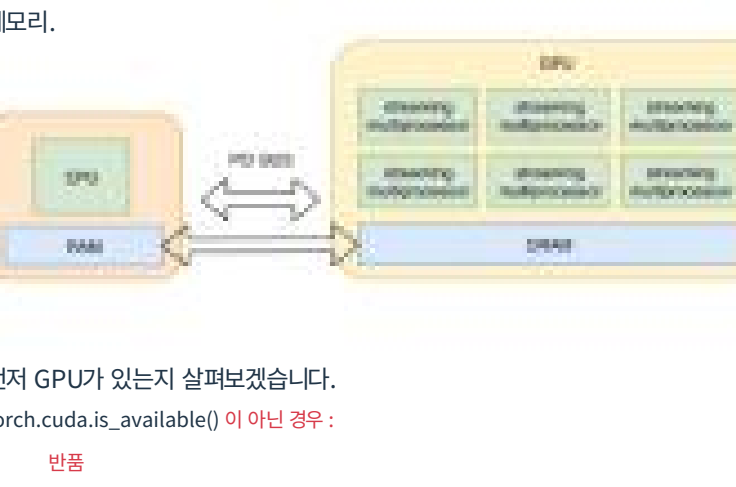
```
x = 토치.텐서([1e-8], dtype=토치.bfloat16) # @x 검사
assert x != 0 # 언더플로가 없습니다!
```

다양한 데이터 유형의 동적 범위와 메모리 사용량을 비교해 보겠습니다.

```
float32_info = torch.finfo(torch.float32) # @inspect float32_info
float16_info = torch.finfo(torch.float16) # @inspect float16_info
bfloat16_info = torch.finfo(torch.bfloat16) # @inspect bfloat16_info
```

에프피8

2022년에 머신 러닝 워크로드에 맞춰 FP8이 표준화되었습니다.
https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/examples/fp8_primer.html



H100은 FP8의 두 가지 변형인 E4M3(범위 [-448, 448]) 및 E5M2([-57344, 57344])를 지원합니다.
참조: [미시케비시우스+ 2022]

훈련에 대한 의미:

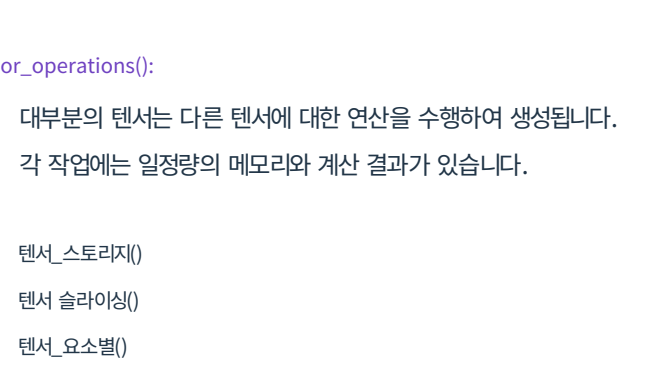
- float32로 학습하는 것은 가능하지만 많은 메모리가 필요합니다.
- fp8, float16, 심지어 bfloat16을 사용하여 훈련하는 것은 위험하며 불안정해질 수 있습니다.
- 해결책(나중에): 혼합 정밀도 훈련을 사용합니다. mixed_precision_training을 참조하세요.

```
def tensors_on_gpus():
```

기본적으로 텐서는 CPU 메모리에 저장됩니다.

```
x = 토치.zeros(32, 32)
x.device == torch.device("cpu")를 주장합니다 .
```

그러나 GPU의 대규모 병렬 처리를 활용하려면 이를 GPU로 옮겨야 합니다.
메모리.



먼저 GPU가 있는지 살펴보겠습니다.

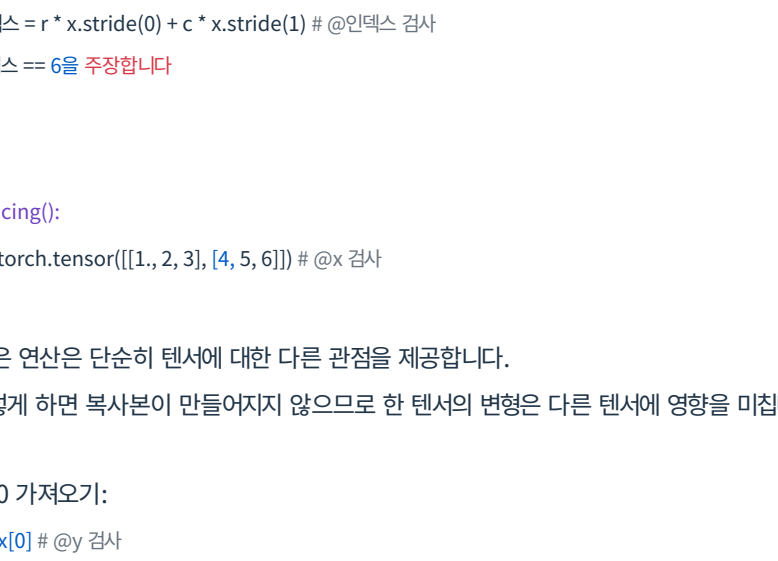
```
torch.cuda.is_available() 이 아닌 경우 :
    반품

num_gpus = torch.cuda.device_count() # @num_gpus 검사
i가 범위(num_gpus) 에 있는 경우 :
    속성 = torch.cuda.get_device_properties(i) # @속성 검사

메모리 할당됨 = torch.cuda.memory_allocated() # @inspect 메모리 할당됨

text("텐서를 GPU 메모리(장치 0)로 이동합니다.")
y = x.to("쿠다:0")
y.device == torch.device("cuda", 0) 을 주장합니다 .

text("또는 GPU에 직접 텐서를 생성합니다:")
z = 토치.zeros(32, 32, 장치="cuda:0")

new_memory_allocated = torch.cuda.memory_allocated() # @inspect new_memory_allocated
메모리_사용됨 = 새_메모리_할당됨 - 메모리_할당됨 # @inspect 메모리_사용됨
assert memory_used == 2 * (32 * 32 * 4) # 4바이트 부동 소수점의 32x32 행렬 2개



def tensor_operations():
```

대부분의 텐서는 다른 텐서에 대한 연산을 수행하여 생성됩니다.
각 작업에는 일정량의 메모리와 계산 결과가 있습니다.

텐서_스토리지()
텐서 슬라이싱()
텐서_요소별()
텐서_매트멀()

```
def tensor_storage():
```

PyTorch의 텐서란 무엇인가요?
PyTorch 텐서는 할당된 메모리에 대한 포인터입니다.
...텐서의 모든 요소에 접근하는 방법을 설명하는 메타데이터가 포함되어 있습니다.



[PyTorch 문서]

```
x = 토치.텐서([
    [0., 1, 2, 3],
    [4, 5, 6, 7],
    [8, 9, 10, 11],
    [12, 13, 14, 15],
])
```

다음 행(dim 0)으로 이동하려면 저장소에서 4개의 요소를 건너뜁니다.

```
x.stride(0) == 4를 주장합니다 .
```

다음 열(dim 1)로 이동하려면 저장소에서 요소 1개를 건너뜁니다.

```
x.stride(1) == 1을 주장합니다 .
```

원소를 찾으려면:

```
r, c = 1, 2
인덱스 = r * x.stride(0) + c * x.stride(1) # @인덱스 검사
인덱스 == 6을 주장합니다


def tensor_slicing():
    x = torch.tensor([[1., 2, 3], [4, 5, 6]]) # @x 검사
```

많은 연산은 단순히 텐서에 대한 다른 관점을 제공합니다.
이렇게 하면 복사본이 만들어지지 않으므로 한 텐서의 변형은 다른 텐서에 영향을 미칩니다.

행 0 가져오기:

```
y = x[0] # @y 검사
torch.equal(y, torch.tensor([1., 2, 3])) 을 주장합니다 .
same_storage(x, y)를 주장합니다 .
```

1열을 가져오세요:

```
y = x[:, 1] # @y 검사
torch.equal(y, torch.tensor([2, 5])) 를 주장합니다 .
same_storage(x, y)를 주장합니다 .
```

2x3 행렬을 3x2 행렬로 보기:

```
y = x.view(3, 2) # @y 검사
torch.equal(y, torch.tensor([[1, 2], [3, 4], [5, 6]]))를 주장합니다 .
same_storage(x, y)를 주장합니다 .
```

행렬을 전치합니다.

```
y = x.transpose(1, 0) # @y 검사
torch.equal(y, torch.tensor([[1, 4], [2, 5], [3, 6]]))를 주장합니다 .
same_storage(x, y)를 주장합니다 .
```

x를 변형하면 y도 변형되는지 확인하세요.

```
x[0][0] = 100 # @x 검사, @y 검사
y[0][0] == 100이라고 주장합니다 .
```

일부 뷰는 연속되지 않은 항목이므로 추가 뷰가 불가능합니다.

```
x = torch.tensor([[1., 2, 3], [4, 5, 6]]) # @x 검사
y = x.transpose(1, 0) # @y 검사
y.is_contiguous() 가 아니라고 주장
노력하다:
    y.view(2, 3)
    거짓을 주장하다
RuntimeError를 e 로 제외 :
    str(e) 에서 "뷰 크기가 입력 텐서의 크기 및 스트라이드와 호환되지 않습니다" 라고 주장합니다 .
```

텐서가 먼저 연속되도록 강제할 수 있습니다.

```
y = x.transpose(1, 0).contiguous().view(2, 3) # @y 검사
same_storage(x, y) 가 아님을 주장합니다.
```

뷰는 무료이지만 복사에는 (추가) 메모리와 컴퓨팅이 모두 필요합니다.

```
def tensor_elementwise():
```

이러한 연산은 텐서의 각 요소에 일부 연산을 적용합니다.
...그리고 같은 모양의 (새로운) 텐서를 반환합니다.

```
x = 토치.텐서([1, 4, 9])
torch.equal(x.pow(2), torch.tensor([1, 16, 81]))를 주장합니다 .
torch.equal(x.sqrt(), torch.tensor([1, 2, 3]))을 주장합니다 .
torch.equal(x.rsqrt(), torch.tensor([1, 1 / 2, 1 / 3]))를 assert합니다 . # i -> 1/sqrt(x_i)

torch.equal(x + x, torch.tensor([2, 8, 18])) 을 주장합니다 .
torch.equal(x * 2, torch.tensor([2, 8, 18])) 을 주장합니다 .
torch.equal(x / 0.5, torch.tensor([2, 8, 18])) 을 주장합니다 .
```

triu는 행렬의 상삼각 부분을 취합니다.

```
x = torch.ones(3, 3).triu() # @x 검사
torch.equal(x, torch.tensor([)를 주장합니다 .
    [1, 1, 1],
    [0, 1, 1],
    [0, 0, 1]],
))
```

이는 M[i, j]가 i의 j에 대한 기여도인 인과적 주의 마스크를 계산하는 데 유용합니다.

```
def tensor_matmul():
```

마지막으로 딥러닝의 핵심은 행렬 곱셈입니다.

```
x = 토치.ones(16, 32)
w = 토치.ones(32, 2)
y = x @ w
y.size() == torch.Size([16, 2])를 주장합니다 .
```

일반적으로 우리는 배치의 모든 예제와 시퀀스의 토큰에 대해 작업을 수행합니다.

batch
sequence

```
x = 토치.ones(4, 8, 16, 32)
w = 토치.ones(32, 2)
y = x @ w
y.size() == torch.Size([4, 8, 16, 2]) 를 주장합니다 .
```

이 경우, 우리는 x 의 처음 2차원 값을 반복 하고 w 로 곱합니다 .

```
def tensor_einops():
    einops_motivation()
```

Einops는 차원에 이름이 지정된 텐서를 조작하기 위한 라이브러리입니다.
이는 아인슈타인의 합산 표기법(Einstein, 1916)에서 영감을 받았습니다.
[에이놉스 튜토리얼]

jaxtyping_basics()
einops_einsum()
einops_reduce()
einops_rearrange()

```
def einops_motivation():
```

기존 PyTorch 코드:

```
x = torch.ones(2, 2, 3) # 배치, 시퀀스, 숨김 @inspect x
y = torch.ones(2, 2, 3) # 배치, 시퀀스, 숨김 @inspect y
z = x @ y.transpose(-2, -1) # 배치, 시퀀스, 시퀀스 @inspect z
```

치수를 엉뚱하게 바꾸기 쉽습니다(-2, -1은 뭐죠?)...

```
def jaxtyping_basics():
```

텐서 차원을 어떻게 추적하나요?

옛날 방식:

```
x = torch.ones(2, 2, 1, 3) # 배치 시퀀스 헤드가 숨겨짐 @inspect x
```

새로운 (잭타이핑) 방식:

```
x: Float[torch.Tensor, "배치 시퀀스 헤드가 숨겨짐"] = torch.ones(2, 2, 1, 3) # @inspect x
```

참고: 이는 단지 문서일 뿐이며 강제력이 없습니다.

```
def einops_einsum():
```

Einsum은 좋은 회계 처리를 갖춘 일반화된 행렬 곱셈입니다.

두 개의 텐서를 정의합니다.

```
x: Float[torch.Tensor, "배치 seq1 숨김"] = torch.ones(2, 3, 4) # @inspect x
y: Float[torch.Tensor, "배치 seq2 숨김"] = torch.ones(2, 3, 4) # @inspect y
```

옛날 방식:

```
z = x @ y.transpose(-2, -1) # 배치, 시퀀스, 시퀀스 @inspect z
```

새로운 (einops) 방식:

```
z = einsum(x, y, "배치 seq1 숨김, 배치 seq2 숨김 -> 배치 seq1 seq2") # @inspect z
```

출력에 이름이 지정되지 않은 차원은 합산됩니다.

또는 ...을 사용하여 여러 차원에 걸친 방송을 표현할 수 있습니다.

```
z = einsum(x, y, "... seq1 숨김, ... seq2 숨김 -> ... seq1 seq2") # @inspect z


def einops_reduce():
```

어떤 연산(예: 합계, 평균, 최대값, 최소값)을 통해 단일 텐서를 줄일 수 있습니다.

```
x: Float[torch.Tensor, "배치 시퀀스 숨김"] = torch.ones(2, 3, 4) # @inspect x
```

옛날 방식:

```
y = x.mean(dim=-1) # @y 검사
```

새로운 (einops) 방식:

```
y = reduce(x, "... 숨김 -> ...", "합계") # @y 검사


def einops_rearrange():
```

때로는 차원이 두 차원을 나타냅니다.
...그리고 그 중 하나를 수술하고 싶어합니다.

```
x: Float[torch.Tensor, "배치 시퀀스 total_hidden"] = torch.ones(2, 3, 8) # @inspect x
```

...여기서 total_hidden은 heads * hidden1 의 평면화된 표현입니다.

```
w: Float[torch.Tensor, "숨겨진1 숨겨진2"] = torch.ones(4, 4)
```

total_hidden을 두 차원 (heads 와 hidden1) 으로 나눕니다 .

```
x = rearrange(x, "... (heads hidden1) ->                                         ... heads hidden1", heads=2) # @inspect x
```

w 로 변환을 수행합니다 .

```
x = einsum(x, w, "... hidden1, hidden1 hidden2 ->                                         ... hidden2") # @inspect x
```

머리 와 hidden2를 다시 합칩니다 .

```
x = rearrange(x, "... heads hidden2 -> ... (heads hidden2)") # @inspect x


def tensor_operations_flops():
```

모든 연산을 살펴보았으니 이제 계산 비용을 살펴보겠습니다.

부동 소수점 연산(FLOP)은 덧셈(x + y)이나 곱셈(xy)과 같은 기본 연산입니다.

몹시 혼란스러운 두 가지 약어(발음은 같음!)

- FLOP: 부동 소수점 연산(수행된 계산의 측정)
- FLOP/s: 초당 부동 소수점 연산(FLOPS라고도 함)으로 속도 측정에 사용됨 하드웨어의.

직관

GPT-3(2020) 훈련은 3.14e23 FLOPs가 소요되었습니 [기사]
GPT-4(2023) 훈련은 2e25 FLOPs를 기록할 것으로 추정됩니다. [미국 행정 명령:
1e26 FLOPs 이상으로 훈련된 모든 기초 모델은 정부에 보고해야 합니다.
(2025년에 폐지됨)

A100은 최대 312 테라플롭/초의 성능을 가지고 있습니다. [사양]

```
a100_flop_per_sec == 312e12를 주장합니다 .
```

H100은 [spec] 없이 50%의 희소성을 갖춘 1979 teraFLOP/s의 최대 성능을 제공합니다.

```
h100_flop_per_sec == 1979e12 / 2 를 주장합니다 .
```

2주 동안 H100 8개:

```
총 플롭 수 = 8 * (60 * 60 * 24 * 7) * h100_flop_per_sec # @inspect 총 플롭 수
```

선형 모델

동기 부여를 위해 선형 모델이 있다고 가정해 보겠습니다.

- 우리는 n개의 포인트를 가지고 있습니다
- 각 점은 d차원입니다
- 선형 모델은 각 d차원 벡터를 ak 출력으로 매핑합니다.

```
torch.cuda.is_available() 인 경우 :
    B = 16384 # 점의 개수
    D = 32768 # 크기
    K = 8192 # 출력 개수
또 다른:
    비 = 1024
    디 = 256
    케이 = 64

장치 = get_device()
x = torch.ones(B, D, 장치=장치)
w = torch.randn(D, K, 장치=장치)
y = x @ w
```

우리는 (i, j, k) 트리플당 하나의 곱셈(x[i][j] * w[j][k])과 하나의 덧셈을 갖습니다.

```
실제 플롭 수 = 2 * B * D * K # @실제 플롭 수 검사
```

다른 작업의 FLOP

- amxn 행렬에 대한 요소별 연산에는 O(mn) FLOP가 필요합니다.
- 두 개의 mxn 행렬을 추가하려면 mn FLOP이 필요합니다.

일반적으로 딥러닝에서 접하는 다른 연산 중 행렬 곱셈만큼 비용이 많이 드는 연산은 없습니다.
충분히 큰 행렬.

해석:

- B는 데이터 포인트의 수입니다
- (DK)는 매개변수의 개수입니다.
- 포워드 패스에 대한 FLOP는 2(토큰 수)(매개변수 수)입니다.

이는 변압기(1차 근사치)에도 일반화되는 것으로 나타났습니다.

FLOP 계산은 실제 시간(초)으로 어떻게 변환되나요?
시간을 정해봅시다!

```
실제 시간 = time_matmul(x, w) # @실제 시간 검사
초당 실제 플롭 수 = 실제 플롭 수 / 실제 시간 # @inspect 초당 실제 플롭 수
```

각 GPU에는 최대 성능을 보고하는 사양서가 있습니다.

- A100 [투기]
- H100 [투기]

FLOP/s는 데이터 유형에 따라 크게 달라집니다!

```
promise_flop_per_sec = get_promised_flop_per_sec(device, x.dtype) # @inspect promise_flop_per_sec
```

모델 FLOP 활용도(MFU)

정의: (실제 FLOP/초) / (약속된 FLOP/초) [통신/오버헤드 무시]

```
mfu = 실제_플롭_당_초 / 약속_플롭_당_초 # @inspect mfu
```

일반적으로 MFU가 0.5 이상이면 매우 좋습니다(matmuls가 우세하면 더 높아질 수 있음)

bfloat16으로 해보겠습니다.

```
x = x.to(토치.bfloat16)
w = w.to(토치.bfloat16)
bf16_실제_시간 = time_matmul(x, w) # @inspect bf16_실제_시간
bf16_actual_flop_per_sec = 실제_플롭_수 / bf16_actual_time # @inspect bf16_actual_flop_per_sec
bf16_promised_flop_per_sec = get_promised_flop_per_sec(장치, x.dtype) # @inspect bf16_promised_flop_per_sec
bf16_mfu = bf16_실제_플롭_당_초 / bf16_약속_플롭_당_초 # @inspect bf16_mfu
```

참고: bfloat16과 float32를 비교하면 실제 FLOP/s가 더 높습니다.
여기의 MFU는 다소 낮은데, 아마도 약속된 FLOP이 다소 낙관적이기 때문일 것입니다.

요약

- 행렬 곱셈이 지배적입니다: (2분) FLOP
- FLOP/s는 하드웨어(H100 >> A100) 및 데이터 유형(bfloat16 >> float32)에 따라 달라집니다.
- 모델 FLOP 활용도(MFU): (실제 FLOP/초) / (약속된 FLOP/초)

```
def gradients_basics():
```

지금까지 우리는 텐서(매개변수 또는 데이터에 해당)를 구성하고 이를 전달했습니다.
작업(전방).
이제 기울기(역방향)를 계산해보겠습니다.

간단한 예로, 간단한 선형 모델을 고려해 보겠습니다.
y = 0.5 (x * w - 5)^2

포워드 패스: 손실 계산

```
x = 토치.텐서([1., 2, 3])
w = torch.tensor([1., 1, 1], requires_grad=True) # 그래디언트를 원함
예측_y = x @ w
손실 = 0.5 * (예상_y - 5).pow(2)
```

역방향 패스: 그래디언트 계산

```
손실.역방향()
loss.grad 가 None 이라고 주장합니다 .
pred_y.grad 가 None 이라고 주장합니다 .
x.grad 가 None 이라고 주장합니다 .
torch.equal(w.grad, torch.tensor([1, 2, 3])) 을 주장합니다 .


def gradients_flops():
```

그래디언트를 계산하기 위해 FLOP을 계산해 보겠습니다.

선형 모델을 다시 살펴보세요

```
torch.cuda.is_available() 인 경우 :
    B = 16384 # 점의 개수
    D = 32768 # 크기
    K = 8192 # 출력 개수
또 다른:
    비 = 1024
    디 = 256
    케이 = 64

장치 = get_device()
x = torch.ones(B, D, 장치=장치)
w1 = torch.randn(D, D, 장치=장치, requires_grad=True)
w2 = torch.randn(D, K, 장치=장치, requires_grad=True)
```

모델: x --w1--> h1 --w2--> h2 -> 손실

```
h1 = x @ w1
h2 = h1 @ w2
손실 = h2.pow(2).mean()
```

전방 FLOP의 수를 기억하세요: tensor_operations_flops

- x[i][j] * w1[j][k]를 곱합니다.
- h1[i][k]에 추가
- h1[i][j] * w2[j][k]를 곱합니다.
- h2[i][k]에 추가

```
num_forward_flops = (2 * B * D * D) + (2 * B * D * K) # @inspect num_forward_flops
```

백워드 패스를 실행하는 데 몇 번의 FLOP이 발생합니까?

```
h1.retain_grad() # 디버깅을 위해
h2.retain_grad() # 디버깅을 위해
손실.역방향()
```

모델 회수: x --w1--> h1 --w2--> h2 -> 손실

- h1.grad = d 손실 / d h1

Machine Translated

- h2.grad = d 손실 / d h2
- w1.grad = d 손실 / d w1
- w2.grad = d 손실 / d w2

매개변수 w2에 초점을 맞추세요.
체인 규칙을 적용합니다.

```
num_backward_flops = 0 # @inspect num_backward_flops
```

w2.grad[j,k] = 합계_i h1[i,j] * h2.grad[i,k]

```
w2.grad.size() == torch.Size([D, K])를 주장합니다 .
h1.size() == torch.Size([B, D])를 주장합니다 .
h2.grad.size() == torch.Size([B, K])를 주장합니다 .
```

각 (i, j, k)에 대해 곱하고 더합니다.

```
num_backward_flops += 2 * B * D * K # @num_backward_flops 검사
```

h1.grad[i,j] = sum_k w2[j,k] * h2.grad[i,k]

```
h1.grad.size() == torch.Size([B, D])를 주장합니다 .
w2.size() == torch.Size([D, K])를 주장합니다 .
h2.grad.size() == torch.Size([B, K])를 주장합니다 .
```

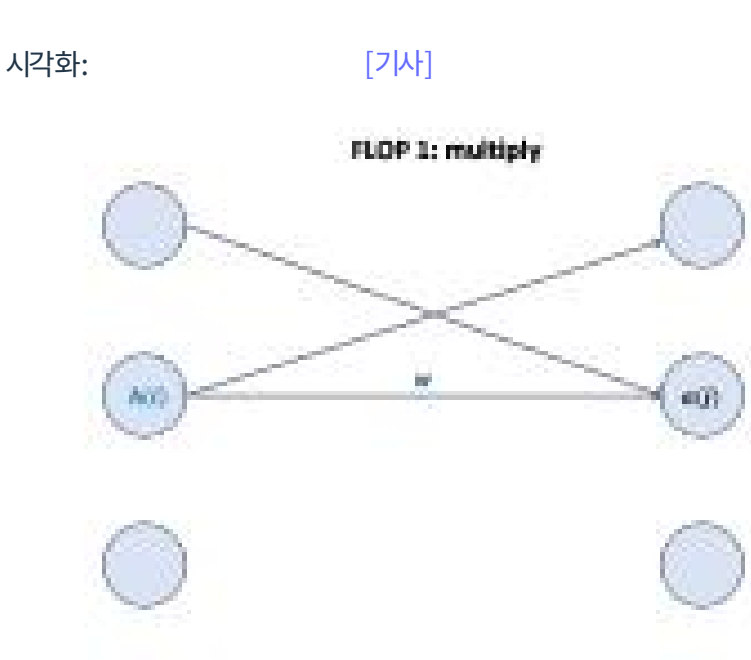
각 (i, j, k)에 대해 곱하고 더합니다.

```
num_backward_flops += 2 * B * D * K # @num_backward_flops 검사
```

이는 w2(D*K 매개변수)에만 해당됩니다.
w1(D*D 매개변수)에도 할 수 있습니다(하지만 x.grad는 필요하지 않습니다).

```
역방향 플롭스 수 += (2 + 2) * B * D * D # @역방향 플롭스 수 검사
```

멋진 그래픽 시각화: [기사]



종합해보면:
- 포워드 패스: 2(# 데이터 포인트)(# 매개변수) FLOP
- 역방향 패스: 4(# 데이터 포인트)(# 매개변수) FLOP
- 총: 6(# 데이터 포인트)(# 매개변수) FLOP

```
def 모델_매개변수():
    입력_차원 = 16384
    출력_차원 = 32
```

모델 매개변수는 PyTorch에 nn.Parameter 객체 로 저장됩니다 .

```
w = nn.Parameter(토치.randn(입력_차원, 출력_차원))
assert isinstance(w, torch.Tensor) # 텐서처럼 동작합니다.
assert type(w.data) == torch.Tensor # 기본 텐서에 접근
```

매개변수 초기화

무슨 일이 일어나는지 지켜보자.

```
x = nn.Parameter(토치.랜드(입력_차원))
출력 = x @ w # @inspect 출력
출력 크기(output_dim) == 토치 크기(output_dim)를 확인합니다 .
```

출력 의 각 요소는 sqrt(input_dim): 18.919979095458984로 확장됩니다.
값이 크면 기울기가 커져 학습이 불안정해질 수 있습니다.

우리는 input_dim에 따라 변하지 않는 초기화를 원합니다 .
그렇게 하려면 간단히 1/sqrt(input_dim)으로 크기를 조정합니다.

```
w = nn.Parameter(토치.randn(입력_차원, 출력_차원) / np.sqrt(입력_차원))
출력 = x @ w # @inspect 출력
```

이제 출력 의 각 요소는 상수입니다: -1.5302726030349731.

상수까지, 이것은 Xavier 초기화입니다. [논문][stackexchange]

더욱 안전하게 하기 위해, 이상치의 발생 가능성을 피하기 위해 정규 분포를 [-3, 3]으로 잘라냅니다.

```
w = nn.Parameter(nn.init.trunc_normal_(torch.empty(입력_차원, 출력_차원), std=1 / np.sqrt(입력_차원), a=-3, b=3))

def custom_model():
```

nn.Parameter를 사용하여 간단한 심층 선형 모델을 구축해 보겠습니다 .

```
D = 64 # 크기
레이어 수 = 2
모델 = Cruncher(dim=D, num_layers=num_layers)

매개변수_크기 = [
    (이름, 매개변수.numel())
    name , param in model.state_dict().items()에 대한
]
매개변수 크기 == [를 주장합니다 .
    ("layers.0.weight", D * D),
    ("layers.1.weight", D * D),
    ("최종 무게", D),
]
num_parameters = get_num_parameters(모델)
매개변수 수 == (D * D) + (D * D) + D를 확인합니다 .
```

모델을 GPU로 옮기는 것을 잊지 마세요.

```
장치 = get_device()
모델 = 모델.to(장치)
```

일부 데이터에 대해 모델을 실행합니다.

```
B = 8 # 배치 크기
x = torch.randn(B, D, 장치=장치)
y = 모델(x)
y.size() == torch.Size([B])를 주장합니다 .


클래스 Linear(nn.Module):
    """단순 선형 레이어."""
    def __init__(자기, 입력_차원: int, 출력_차원: int):
        슈퍼().__init__()
        자기.가중치 = nn.Parameter(torch.randn(입력_차원, 출력_차원) / np.sqrt(입력_차원))

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x @ self.weight 를 반환합니다 .


클래스 Cruncher(nn.Module):
    def __init__(자기 자신, dim: int, num_layers: int):
        슈퍼().__init__()
        self.layers = nn.모듈리스트([
            선형(dim, dim)
            i가 범위(num_layers) 에 있는 경우
        ])
        self.final = Linear(dim, 1)

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        # 선형 레이어 적용
        B, D = x.크기()
        self.layers 의 레이어 에 대해 :
            x = 레이어(x)

        # 최종 헤드 적용
        x = 자기최종(x)
        x.size() == torch.Size([B, 1]) 를 주장합니다 .

        # 마지막 차원을 제거합니다
        x = x.압축(-1)
        x.size() == torch.Size([B])를 주장합니다 .

        x를 반환


def get_batch(데이터: np.array, 배치 크기: int, 시퀀스 길이: int, 장치: str) -> torch.Tensor:
```

샘플 배치_크기의 무작위 위치를 데이터에 넣습니다.

```
시작 인덱스 = torch.randint(len(데이터) - 시퀀스 길이, (배치 크기,))
시작 인덱스.크기() == 토치.크기([배치_크기])를 확인합니다 .
```

데이터 인덱스.

```
x = torch.tensor([[시작 인덱스 의 시작 에 대한 데이터[시작:시작 + 시퀀스_길이] ])
x.size() == torch.Size([배치_크기, 시퀀스_길이])를 주장합니다 .
```

고정된 메모리

기본적으로 CPU 텐서는 페이지 메모리에 있습니다. 명시적으로 고정할 수 있습니다.

```
torch.cuda.is_available() 인 경우 :
    x = x.핀_메모리()
```

이를 통해 CPU에서 GPU로 x를 비동기적으로 복사할 수 있습니다 .

```
x = x.to(장치, 비차단=참)
```

이를 통해 두 가지 작업을 병렬로 수행할 수 있습니다(여기서는 수행하지 않음):
- 다음 데이터 배치를 CPU로 가져옵니다.
- GPU에서 x를 처리합니다 .

[기사]
[기사]

```
x를 반환


def note_about_randomness():
```

무작위성은 매개변수 초기화, 드롭아웃, 데이터 정렬 등 여러 곳에서 나타납니다.
재현성을 위해 무작위성을 사용할 때마다 항상 다른 무작위 시드를 전달하는 것이 좋습니다.
결정론은 디버깅할 때 특히 유용하며, 버그를 찾아낼 수 있습니다.

안정을 위해 무작위 시드를 설정할 수 있는 세 곳이 있는데, 모두 한꺼번에 설정하는 것이 좋습니다.

```
# 토치
시드 = 0
torch.manual_seed(시드)

# 넘파이
numpy를 np 로 가져오기
np.random.seed(시드)

#파이썬
무작위로 가져오기
랜덤.시드(시드)


def 데이터_로딩():
```

언어 모델링에서 데이터는 정수 시퀀스(토큰화기의 출력)입니다.

이를 numpy 배열로 직렬화하는 것이 편리합니다(토큰화기를 통해 수행).

```
원래 데이터 = np.배열([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], dtype=np.int32)
orig_data.tofile("data.npy")
```

이를 numpy 배열로 다시 로드할 수 있습니다.
전체 데이터를 한 번에 메모리에 로드하고 싶지 않습니다(LLaMA 데이터는 2.8TB입니다).
memmap을 사용하여 액세스된 부분만 메모리에 지연 로드할 수 있습니다.

```
데이터 = np.memmap("data.npy", dtype=np.int32)
np.array_equal(데이터, orig_data)를 주장합니다 .
```

데이터 로더는 훈련 을 위한 일련의 시퀀스를 생성합니다.

```
B = 2 # 배치 크기
L = 4 # 시퀀스의 길이
x = get_batch(데이터, 배치 크기=B, 시퀀스 길이=L, 장치=get_device())
x.size() == torch.Size([B, L])를 주장합니다 .


클래스 SGD(torch.optim.Optimizer):
    def __init__(self, 매개변수: Iterable[nn.Parameter], lr: float = 0.01):
        super(SGD, self).__init__(매개변수, dict(lr=lr))

    def step(자기 자신):
        self.param_groups 의 그룹 에 대해 :
            lr = 그룹["lr"]
            그룹["params"] 의 p 에 대해 :
                학위 = p.degree.data
                p.data -= lr * grad


클래스 AdaGrad(torch.optim.Optimizer):
    def __init__(self, 매개변수: Iterable[nn.Parameter], lr: float = 0.01):
        super(AdaGrad, self).__init__(매개변수, dict(lr=lr))

    def step(자기 자신):
        self.param_groups 의 그룹 에 대해 :
            lr = 그룹["lr"]
            그룹["params"] 의 p 에 대해 :
                # 최적화기 상태
                상태 = self.state[p]
                학위 = p.degree.data

                # 제곱 기울기 얻기 g2 = sum_{i<t} g_i^2
                g2 = state.get("g2", torch.zeros_like(grad))

                # 최적화기 상태 업데이트
                g2 += 토치.제곱(그라디언트)
                상태["g2"] = g2

                # 매개변수 업데이트
                p.data -= lr * grad / torch.sqrt(g2 + 1e-5)


def optimizer():
```

우리의 심층 선형 모델을 떠올려 보세요.

```
비 = 2
디 = 4
레이어 수 = 2
모델 = Cruncher(dim=D, num_layers=num_layers).to(get_device())
```

AdaGrad 최적화기를 정의해 보겠습니다.
- 모멘텀 = SGD + 그래디언트의 지수 평균
- AdaGrad = SGD + grad^2 평균
- RMSProp = AdaGrad + grad^2의 지수 평균
- 아담 = RMSProp + 모멘텀

AdaGrad: https://www.jmlr.org/papers/volume12/duchi11a/duchi11a.pdf

```
최적화기 = AdaGrad(모델.매개변수(), lr=0.01)
상태 = 모델.state_dict() # @상태 검사
```

기울기 계산

```
x = 토치.randn(B, D, 장치=get_device())
y = 토치.텐서([4., 5.], 장치=get_device())
예측_y = 모델(x)
손실 = F.mse_loss(입력=예측_y, 목표=y)
손실.역방향()
```

한 걸음 내딛다

```
옵티마이저.단계()
상태 = 모델.state_dict() # @상태 검사
```

메모리 확보(선택 사항)

```
optimizer.zero_grad(set_to_none=True)
```

메모리

```
# 매개변수
num_parameters = (D * D * num_layers) + D # @inspect num_parameters
assert num_parameters == get_num_parameters(모델)

# 활성화
num_activations = B * D * num_layers # @inspect num_activations

# 그라디언트
num_gradients = num_parameters # @inspect num_gradients

# 최적화기 상태
num_optimizer_states = num_parameters # @inspect num_optimizer_states

# 모든 것을 함께 놓고 float32를 가정합니다.
총 메모리 = 4 * (매개변수 수 + 활성화 수 + 경사도 수 + 최적화 상태 수) # @inspect 총 메모리
```

(한 단계 동안) 계산

```
플롭스 = 6 * B * 매개변수 수 # @inspect 플롭스
```

변압기

변압기에 대한 회계 처리는 좀 더 복잡하지만, 개념은 같습니다.
과제 1에서 그걸 요구할 겁니다.

Transformer 학습을 위한 메모리 사용을 설명하는 블로그 게시물 [기사] [기사]
변압기에 대한 FLOP을 설명하는 블로그 게시물:

```
def train_loop():
```

가중치 (0, 1, 2, ..., D-1)을 갖는 선형 함수로부터 데이터를 생성합니다.

```
디 = 16
true_w = torch.arange(D, dtype=torch.float32, 장치=get_device())
def get_batch(B: int) -> 튜플[torch.Tensor, torch.Tensor]:
    x = 토치.randn(B, D).to(get_device())
    참_y = x @ 참_w
    (x, true_y) 를 반환합니다 .
```

기본적인 실행을 해보자

```
train("simple", get_batch, D=D, num_layers=0, B=4, num_train_steps=10, lr=0.01)
```

하이퍼파라미터 튜닝을 해보세요

```
train("simple", get_batch, D=D, num_layers=0, B=4, num_train_steps=10, lr=0.1)


def train(이름: str, get_batch,
          D: int, num_layers: int,
          B: int, num_train_steps: int, lr: float):
    모델 = Cruncher(dim=D, num_layers=0).to(get_device())
    최적화기 = SGD(모델.매개변수(), lr=0.01)

    t가 범위(num_train_steps) 에 있는 경우 :
        # 데이터 가져오기
        x, y = get_batch(B=B)

        # 전방(손실 계산)
        예측_y = 모델(x)
        손실 = F.mse_loss(예측_y, y)

        # 역방향(그래디언트 계산)
        손실.역방향()

        # 매개변수 업데이트
        옵티마이저.단계()
        optimizer.zero_grad(set_to_none=True)


def 체크포인팅():
```

언어 모델을 훈련하는 데는 시간이 오래 걸리고 확실히 충돌이 일어날 것입니다.
진행 상황을 모두 잃고 싶지는 않을 겁니다.

학습하는 동안 모델과 최적화기 상태를 주기적으로 디스크에 저장하는 것이 유용합니다.

```
모델 = Cruncher(dim=64, num_layers=3).to(get_device())
최적화기 = AdaGrad(모델.매개변수(), lr=0.01)
```

체크포인트를 저장하세요:

```
체크포인트 = {
    "모델": model.state_dict(),
    "최적화기": optimizer.state_dict(),
}
torch.save(체크포인트, "모델_체크포인트.pt")
```

체크포인트를 로드합니다:

```
로드된 체크포인트 = torch.load("모델 체크포인트.pt")


def 혼합_정밀_훈련():
```

데이터 유형(float32, bfloat16, fp8)을 선택하면 그에 따른 장단점이 있습니다.
- 더 높은 정밀도: 더 정확하고 안정적이며, 더 많은 메모리, 더 많은 컴퓨팅
- 낮은 정밀도: 정확도/안정성 저하, 메모리 부족, 컴퓨팅 부족

어떻게 하면 두 세계의 장점을 모두 얻을 수 있을까?

해결 방법: 기본적으로 float32를 사용하지만 가능하면 {bfloat16, fp8}을 사용합니다.

구체적인 계획:
- 전방 패스(활성화)에는 {bfloat16, fp8}을 사용합니다.
- 나머지(매개변수, 그래디언트)에는 float32를 사용합니다.

- 혼합 정밀 훈련 [미시케비시우스+ 2017]

Pytorch에는 AMP(자동 혼합 정밀도) 라이브러리가 있습니다.
https://pytorch.org/docs/stable/amp.html
https://docs.nvidia.com/deeplearning/performance/mixed-precision-training/

NVIDIA의 Transformer Engine은 선형 레이어에 대해 FP8을 지원합니다.
훈련 전반에 걸쳐 FP8을 널리 활용하세요 [펭+ 2023]

```
############################################################

def get_memory_usage(x: torch.Tensor):
    x.numel() * x.element_size()를 반환합니다 .


def get_promised_flop_per_sec(장치: str, dtype: torch.dtype) -> float:
    """`dtype`에서 작동하는 `device`에 대한 최대 FLOP/s를 반환합니다."""
    torch.cuda.is_available() 이 아닌 경우 :
        CUDA 장치를 사용할 수 없으므로 FLOP/s를 얻을 수 없습니다.
        1을 반환하다
    속성 = torch.cuda.get_device_properties(장치)

    properties.name 에 "A100"이 있는 경우 :
        # https://www.nvidia.com/content/dam/en-zz/솔루션/데이터-센터/a100/pdf/nvidia-a100-데이터시트-us-nvidia-1758950-r4-웹.pdf")
        dtype == torch.float32 인 경우 :
            19.5e12를 반환합니다
        dtype 이 (torch.bfloat16, torch.float16) 인 경우 :
            312e12를 반환합니다
        ValueError(f"알 수 없는 dtype: {dtype}")를 발생시킵니다 .

    properties.name 에 "H100"이 있는 경우 :
        # https://resources.nvidia.com/en-us-tensor-core/nvidia-tensor-core-gpu-데이터시트")
        dtype == torch.float32 인 경우 :
            67.5e12를 반환합니다
        dtype 이 (torch.bfloat16, torch.float16) 인 경우 :
            return 1979e12 / 2 # 1979는 sparse용 이라고 dense는 그 절반입니다.
        ValueError(f"알 수 없는 dtype: {dtype}")를 발생시킵니다 .

    ValueError(f"알 수 없는 장치: {device}")를 발생시킵니다 .


def same_storage(x: torch.Tensor, y: torch.Tensor):
    x.untyped_storage().data_ptr() == y.untyped_storage().data_ptr()를 반환합니다 .


def time_matmul(a: 토치.텐서, b: 토치.텐서) -> float:
    """`a @ b`를 수행하는 데 필요한 초 수를 반환합니다."""

    # 이전 CUDA 스레드가 완료될 때까지 기다리세요
    torch.cuda.is_available() 인 경우 :
        토치.쿠다.동기화()

    def run():
        # 작업을 수행하다
        아 @ 비

        # CUDA 스레드가 완료될 때까지 기다리세요
        torch.cuda.is_available() 인 경우 :
            토치.쿠다.동기화()

    # 작업 `num_trials` 시간을 측정합니다.
    시도 횟수 = 5
    총 시간 = timeit.timeit(실행, 횟수=시도 횟수)

    총 시간 / 시도 횟수를 반환합니다 .


def get_num_parameters(모델: nn.Module) -> int:
    model.parameters() 의 param 에 대해 sum(param.numel())을 반환합니다 .

def get_device(인덱스: int = 0) -> torch.device:
    "가능하면 GPU를 사용하고, 그렇지 않으면 CPU를 사용하세요."
    torch.cuda.is_available() 인 경우 :
        torch.device(f"cuda:{index}")를 반환합니다 .
    또 다른:
        torch.device("cpu")를 반환합니다 .

__name__ == "__main__" 인 경우 :
    기본()
```